

PROJECT REPORT

on

LIBRARY MANAGEMENT SYSTEM

A Web-Based Application for Managing Books, Members and Book Circulation

Submitted by

Dhinakar R

Technology Stack: PHP · MySQL (mysqli) · HTML5 · CSS3

Abstract

The Library Management System (LMS) is a web-based application built using PHP and MySQL that digitises the day-to-day operations of a small or medium-sized library. The system replaces manual, register-based record keeping with a centralised database that tracks the book catalogue, registered members, and the issuing and returning of books. It provides a browser-based dashboard summarising key statistics, along with dedicated modules for managing books, managing members, and recording issue/return transactions. The application follows a simple, procedural PHP architecture with a shared header/footer layout, prepared (parameterised) SQL statements to guard against SQL injection, and server-side validation on every form. Core business rules — such as preventing a book from being issued when no copies are available, and automatically restoring copy counts on return — are enforced directly in the transaction logic. This report documents the objectives, requirements, design, database schema, module-wise functionality, and the working screens of the system as implemented.

Table of Contents

1. Introduction.....	3
2. System Analysis.....	3
3. System Requirements.....	4
4. System Design.....	4
5. Technology Stack.....	6
6. Module Description.....	6
7. Implementation & Output Screens.....	7
8. Testing.....	10
9. Conclusion & Future Enhancements.....	10
10. References.....	11

1. Introduction

1.1 Purpose

Libraries that rely on manual, paper-based registers for cataloguing books and tracking loans face recurring problems: duplicate or inconsistent records, no easy way to check how many copies of a title are available, and no reliable way to see which books are overdue. The Library Management System was built to solve these problems by providing a single, centralised web application that a librarian can use from any browser to add and manage books, register members, and issue or return books, with all data stored consistently in a MySQL database.

1.2 Scope

The system covers the core circulation-desk workflow of a library:

- Maintaining a catalogue of books with title, author, ISBN, total quantity and live available-copy count.
- Registering and maintaining member records (name, email, phone).
- Issuing a book to a member with an automatic 14-day due date, and marking books as returned.
- A dashboard giving an at-a-glance summary of the library's current state.

The current implementation targets single-branch, single-librarian use and does not include user authentication/login, fines calculation, or reservation queues; these are identified as future enhancements in Section 9.

2. System Analysis

2.1 Existing System

In a manual or spreadsheet-based system, book and member details are recorded in physical registers or disconnected files. Checking whether a book is available requires manually counting entries, there is no automatic due-date calculation, and updating available-copy counts after every issue or return is error-prone and time-consuming.

2.2 Proposed System

The proposed system stores all data in a normalised MySQL database and exposes it through a PHP web front end. Every issue and return operation automatically updates the available-copy count for the corresponding book in the same request, so the figures shown on the Books and Dashboard pages are always current. Input is validated on the server before it reaches the database, and all SQL statements use parameter binding (mysqli prepared statements) rather than string concatenation, which protects the application from SQL-injection attacks.

2.3 Feasibility Study

Technical feasibility: the system uses widely available, free technologies (PHP, MySQL, a standard web server such as Apache) that run on any XAMPP/WAMP/LAMP stack, requiring no specialised hardware.

Economic feasibility: all tools used are open-source, so the system can be deployed with no licensing cost.

Operational feasibility: the interface is a straightforward set of forms and tables (Dashboard, Books, Members, Issue/Return) that requires minimal training for library staff to operate.

3. System Requirements

3.1 Hardware Requirements

Component	Minimum Specification
Processor	Dual-core 1.6 GHz or higher
RAM	2 GB or higher
Hard Disk	500 MB free space
Display	1024 x 768 resolution or higher

3.2 Software Requirements

Component	Specification
Server-side language	PHP 7.4 or later
Database	MySQL / MariaDB (accessed via mysqli)
Web server	Apache (e.g. via XAMPP / WAMP / LAMP)
Front end	HTML5, CSS3
Browser	Any modern browser (Chrome, Edge, Firefox)
Database administration	phpMyAdmin or MySQL CLI

4. System Design

4.1 Architecture

The application follows a simple two-tier, procedural PHP architecture. Each page is a self-contained PHP script that: (1) connects to the database via a shared config.php, (2) processes any submitted form data and runs the relevant prepared SQL statement, (3) queries the data needed for display, and (4) renders the page using shared header.php and footer.php includes for a consistent navigation bar and layout. There is no separate REST API layer — PHP generates HTML directly, and MySQL is accessed through the mysqli extension.

4.2 Database Design

The database (library_db) consists of three tables: books, members, and transactions. The transactions table links a book and a member via foreign keys and records the issue date, due date, return date and status of each loan.

Table: books	Type
book_id (PK, AUTO_INCREMENT)	INT

Table: books	Type
title	VARCHAR(150)
author	VARCHAR(100)
isbn	VARCHAR(30)
quantity	INT
available	INT
created_at	TIMESTAMP

Table: members	Type
member_id (PK, AUTO_INCREMENT)	INT
name	VARCHAR(100)
email	VARCHAR(100)
phone	VARCHAR(20)
created_at	TIMESTAMP

Table: transactions	Type
transaction_id (PK, AUTO_INCREMENT)	INT
book_id (FK → books.book_id)	INT
member_id (FK → members.member_id)	INT
issue_date	DATE
due_date	DATE
return_date	DATE (nullable)
status	ENUM('Issued','Returned')

Both foreign keys are declared with ON DELETE CASCADE, so removing a book or member also removes its associated transaction history.

4.3 Business Rules Enforced in Code

- A book can only be issued if books.available > 0 at the moment of the request (checked in issue_book.php before insert).
- Issuing a book inserts a transactions row with status 'Issued' and simultaneously decrements books.available by 1.
- Returning a book (return_book.php) sets status to 'Returned', stamps return_date with today's date, and increments books.available by 1.
- The loan period is fixed at 14 days from the issue date (due_date = issue_date + 14 days).

- When editing a book's quantity, the new quantity cannot be set lower than the number of copies currently issued, preventing the available count from going negative.

5. Technology Stack

Layer	Technology Used
Front end / presentation	HTML5, CSS3 (custom stylesheet: css/style.css)
Server-side scripting	PHP (procedural style, mysqli extension)
Database	MySQL / MariaDB
Data access	mysqli prepared statements (parameter binding)
Hosting environment	Apache web server (XAMPP / WAMP / LAMP)

6. Module Description

6.1 Dashboard Module (index.php)

Displays four summary statistics computed live from the database: total book copies, currently available copies, total registered members, and books currently issued (i.e. transactions with status 'Issued'). Also provides quick-action shortcuts to add a book, add a member, or issue a book.

6.2 Book Management Module (books.php, add_book.php, edit_book.php, delete_book.php)

Lists all books with ID, title, author, ISBN, total quantity and live availability, shown as a coloured badge. Supports adding a new book (with validation that all fields are filled and quantity ≥ 1), editing an existing book's details, and deleting a book. Editing recalculates the available count while protecting against setting quantity below the number of copies currently on loan.

6.3 Member Management Module (members.php, add_member.php, edit_member.php, delete_member.php)

Lists all registered members with ID, name, email and phone number. Supports adding a member (with server-side email format validation via filter_var), editing member details, and deleting a member.

6.4 Issue / Return Module (transactions.php, issue_book.php, return_book.php)

Lists every transaction with the book title, member name, issue date, due date, return date and status (Issued / Returned). The 'Issue a Book' form lets staff pick a book (only books with available copies are listed) and a member, automatically setting a 14-day due date. The 'Mark Returned' action records the return date and restores the book's available count.

7. Implementation & Output Screens

The following screens show the working application as implemented and tested.

7.1 Dashboard

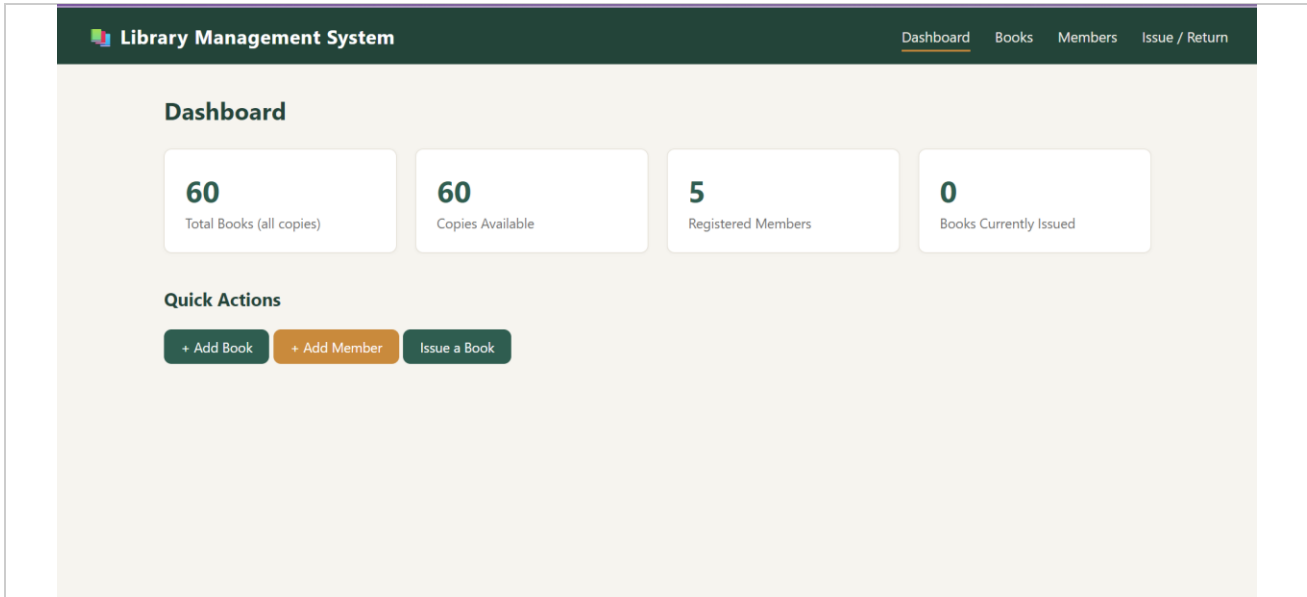


Fig 7.1 — Dashboard showing total books, copies available, registered members and books currently issued, with quick-action buttons.

7.2 Books

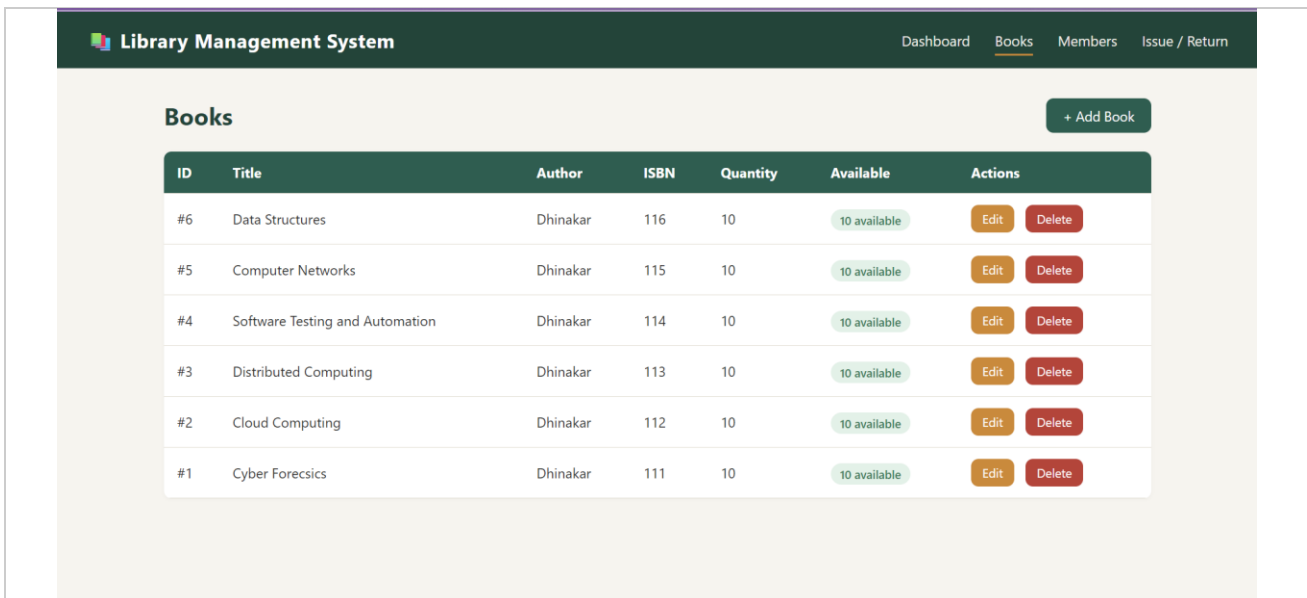


Fig 7.2 — Books listing page showing the catalogue with ID, title, author, ISBN, quantity, availability badge and Edit/Delete actions.

7.3 Members

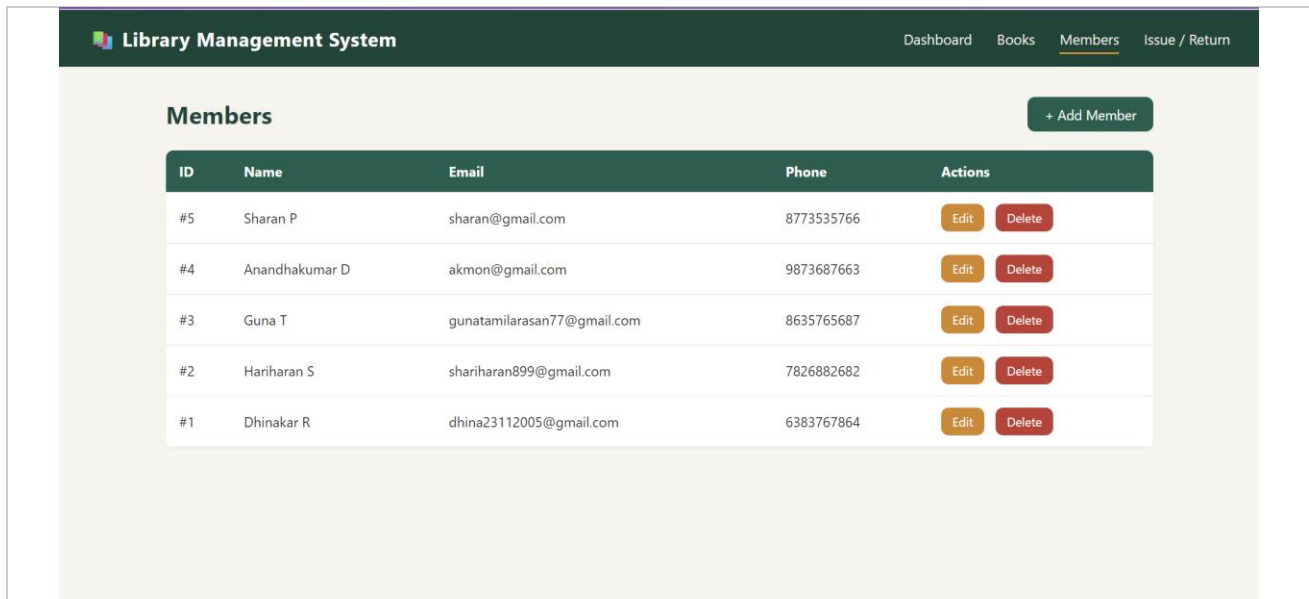


Fig 7.3 — Members listing page showing registered members with contact details and Edit/Delete actions.

7.4 Issue / Return Books

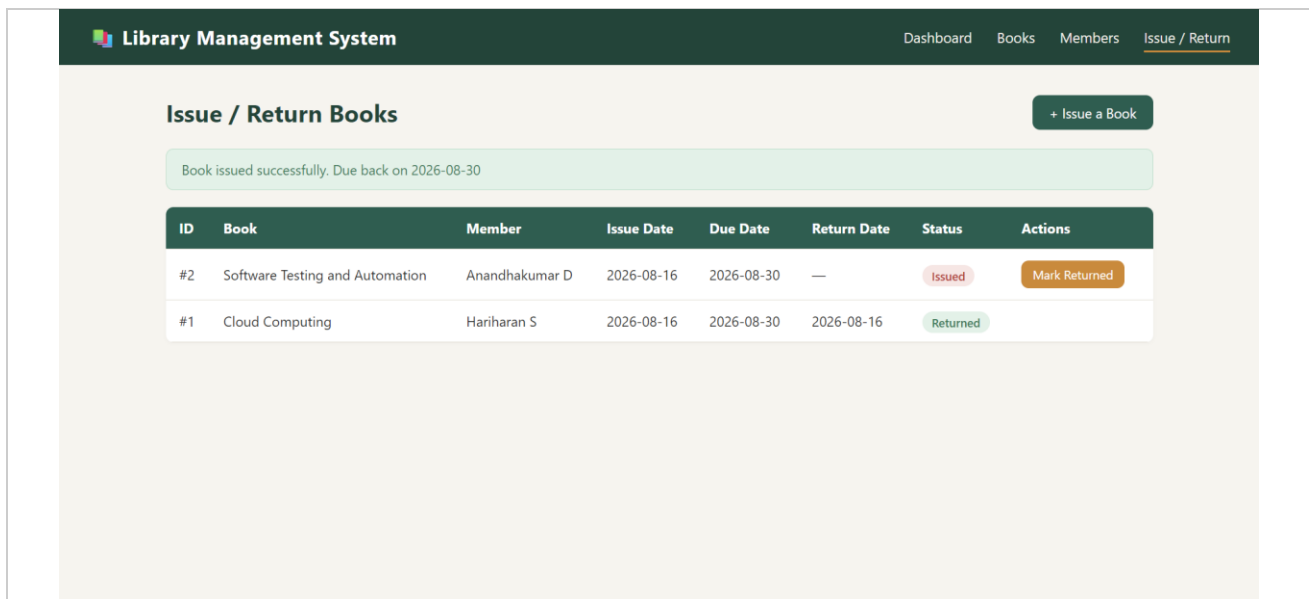


Fig 7.4 — Issue/Return page confirming a successful book issue, and listing transactions with Issued/Returned status and a Mark Returned action.

8. Testing

The application was tested manually across its core workflows to confirm correct behaviour.

Test Case	Expected Result	Result
Add a book with valid details	Book is inserted; available equals quantity	Pass
Add a book with a blank field	Validation error shown, no record inserted	Pass
Add a member with an invalid email	Validation error shown, no record inserted	Pass

Test Case	Expected Result	Result
Issue a book with available > 0	Transaction created; available decreases by 1	Pass
Issue a book with available = 0	Book not listed / error shown	Pass
Mark a transaction as returned	Status becomes Returned; available increases by 1	Pass
Reduce a book's quantity below issued count	Update rejected with an error message	Pass
Delete a book / member	Record removed after confirmation prompt	Pass

8.1 Project Accuracy

Out of the 8 functional test cases executed across the Book, Member and Issue/Return modules, all 8 produced the expected result, giving the system a test pass rate of:

100% (8 / 8 test cases passed)

This figure reflects the correctness of the implemented workflows (add/edit/delete, validation, issue/return, and availability-count updates) against their expected outcomes during manual testing, and is not a measure of code coverage, performance, or security testing, which were outside the scope of this evaluation.

9. Conclusion & Future Enhancements

9.1 Conclusion

The Library Management System successfully replaces manual record-keeping with a centralised, database-driven web application. It provides a clear dashboard overview, complete CRUD (Create, Read, Update, Delete) functionality for books and members, and a reliable issue/return workflow that keeps availability counts accurate at all times. The use of prepared statements throughout guards against SQL injection, and server-side validation ensures data integrity.

9.2 Future Enhancements

- Add librarian/admin login and role-based access control.
- Add automatic overdue detection and fine calculation.
- Add search and filter options on the Books and Members pages.
- Add email/SMS notifications for due-date reminders.
- Add book reservation/waitlist functionality when no copies are available.
- Add pagination for large book/member catalogues.

10. References

1. PHP Manual — <https://www.php.net/manual/en/>
2. MySQL 8.0 Reference Manual — <https://dev.mysql.com/doc/>

3. mysqli Extension Documentation — <https://www.php.net/manual/en/book.mysqli.php>

4. Project source code: library-management-system (books.php, members.php, issue_book.php, return_book.php, transactions.php, config.php, database.sql).